

PHASE 3: SECURITY HARDENING & CLOUDFRONT INTEGRATION

Technical Implementation and Capstone Report

OVERVIEW

This phase focused on strengthening the security posture of the deployed AWS infrastructure while introducing a Content Delivery Network (CDN) layer through Amazon CloudFront. The primary objectives were to enforce HTTPS across all client communications, restrict direct access to backend resources, implement least-privilege IAM permissions, secure Amazon S3 access using Origin Access Control (OAC), and validate the complete HTTPS request flow from the user’s browser to the backend EC2 instances.

TABLE OF CONTENTS

Section	Description
1	CloudFront Distribution Setup
2	ACM Certificate Attachment
3	CloudFront Behavior Configuration
4	Security Group Restrictions
5	IAM Least Privilege Policies
6	S3 Bucket Policy with Origin Access Control (OAC)
7	Security Group and Port Audit
8	End-to-End HTTPS Flow Testing

1. CLOUDFRONT DISTRIBUTION SETUP

Objective

Create a global Content Delivery Network (CDN) layer using Amazon CloudFront with the Application Load Balancer (ALB) configured as the origin.

Implementation Steps

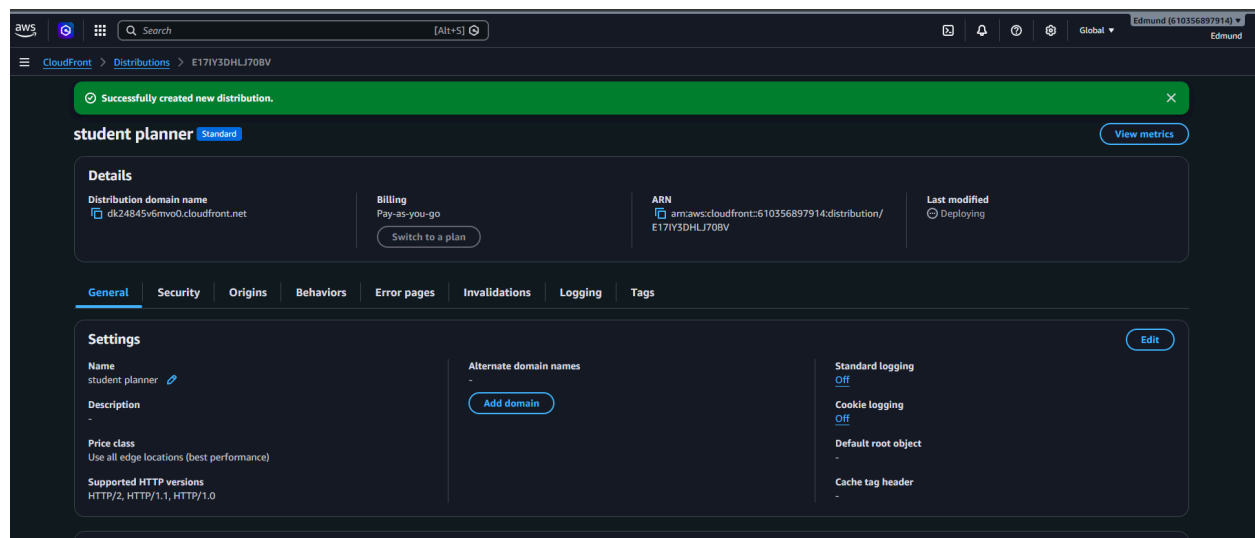
1. Navigated to **AWS Management Console** → **CloudFront** → **Create Distribution**.
2. Configured the **Origin Domain** using the public DNS name of the Application Load Balancer.
3. Set the **Origin Protocol Policy** to **HTTPS Only** to ensure secure communication between CloudFront and the origin.
4. Retained the default cache behavior during initial deployment.
5. Deployed the CloudFront distribution and verified successful propagation across AWS edge locations.

Outcome

The CloudFront distribution was successfully deployed and configured to route traffic securely through the Application Load Balancer.

Evidence

Figure 1: CloudFront Distribution Origin Configuration



2. ACM CERTIFICATE ATTACHMENT

Objective

Attach an SSL/TLS certificate issued through AWS Certificate Manager (ACM) to the CloudFront distribution to enable HTTPS for the custom domain.

Implementation Steps

1. Opened the target CloudFront distribution.
2. Navigated to the **General Settings** section.
3. Selected the ACM certificate associated with the custom domain.
4. Configured the minimum TLS version to **TLSv1.2_2021**.
5. Saved changes and waited for CloudFront deployment completion.

Important Consideration

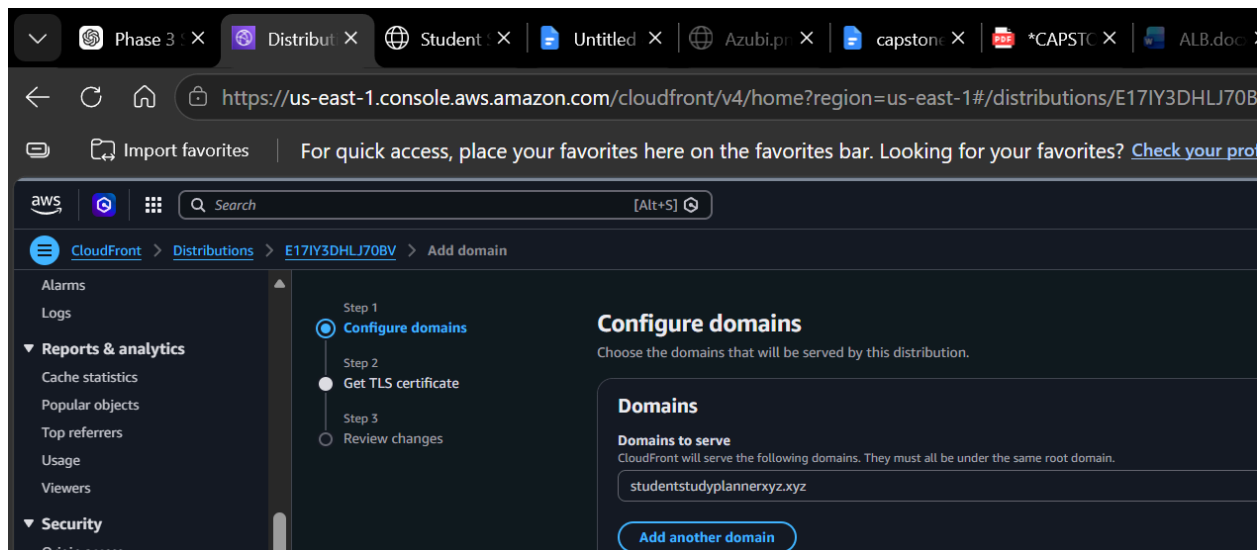
AWS requires all ACM certificates used by CloudFront distributions to be created in the **US East (N. Virginia) Region (us-east-1)** regardless of the region hosting the application resources.

Outcome

HTTPS was successfully enabled on the custom domain using the ACM certificate.

Evidence

Figure 2: CloudFront Distribution with ACM Certificate Attached



Get TLS certificate

TLS certificate

Info

Refresh certificates

Transport layer security (TLS) encrypts communication to and from your domain. You must have a TLS certificate with AWS Certificate Manager (ACM) to use CloudFront.

Available certificates

Info

These certificates cover the domains that will be served by this distribution.

☒ studentstudyplannerxyz.xyz (0cce8ad9-3f89-4a94-b4dd-6ed6ffbb7a37)

☐ studentstudyplannerxyz.xyz (c1f265a5-0203-438b-ad40-278229a84959)

☐ Create a new certificate

Certificate details

View in AWS Certificate Manager

ARN

am:aws:acm:us-east-1:610356897914:certificate/0cce8ad9-3f89-4a94-b4dd-6ed6ffbb7a37

Covered domains

studentstudyplannerxyz.xyz
*.studentstudyplannerxyz.xyz

Source

Amazon

Cancel

Previous

Next

3. CLOUDFRONT BEHAVIOR CONFIGURATION

Objective

Configure CloudFront behaviors to enforce HTTPS and optimize content delivery for static assets.

Implementation Steps

HTTPS Enforcement

1. Opened the **Behaviors** tab within the CloudFront distribution.
2. Edited the default behavior.
3. Set **Viewer Protocol Policy** to:
Redirect HTTP to HTTPS

Static Content Optimization

1. Created a separate cache behavior for:
`/static/*`
2. Attached the AWS-managed cache policy:
CachingOptimized
3. Configured dynamic content to use:
CachingDisabled

Origin Request Policy

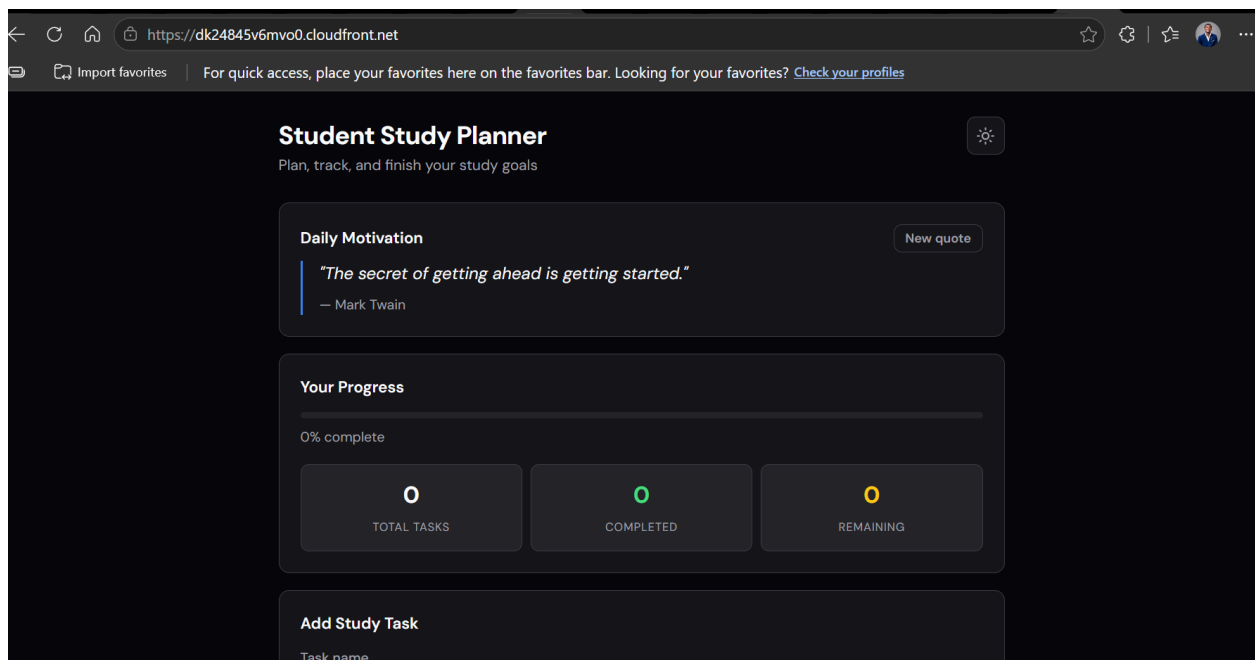
Attached:

AllViewerExceptHostHeader

This policy forwards necessary request information while preventing host header conflicts.

Outcome

All incoming HTTP requests are automatically redirected to HTTPS, while static content benefits from CloudFront edge caching.



4. SECURITY GROUP RESTRICTIONS

Objective

Restrict access to the Application Load Balancer and backend resources using AWS-managed CloudFront prefix lists.

Implementation Steps

1. Navigated to:
EC2 → Security Groups
2. Selected the Application Load Balancer Security Group.

3. Removed unrestricted inbound rules allowing traffic from:
0.0.0.0/0
4. Added inbound rules permitting access only from:
com.amazonaws.global.cloudfront.origin-facing
5. Configured:
 - Port 80 (HTTP)
 - Port 443 (HTTPS)
6. Verified that backend EC2 instances only accept traffic originating from the ALB security group.

Security Improvement Summary

Rule	Before	After
HTTP (80)	Public (0.0.0.0/0)	CloudFront Prefix List Only
HTTPS (443)	Public (0.0.0.0/0)	CloudFront Prefix List Only
EC2 Access	Public Exposure	ALB Security Group Only

Outcome

Direct public access to backend resources was eliminated, ensuring all external traffic passes through CloudFront.

5. IAM LEAST PRIVILEGE POLICIES

Objective

Apply least-privilege access controls across AWS services.

Implementation Steps

EC2 IAM Role

Configured permissions limited to:

- s3:GetObject
- logs:CreateLogGroup
- logs:CreateLogStream
- logs:PutLogEvents

CloudFront OAC Permissions

Restricted S3 object access to requests originating exclusively from the approved CloudFront distribution.

CI/CD Deployment User

Granted only required permissions for:

- CloudFront invalidations
- S3 object uploads
- EC2 metadata access

Example EC2 IAM Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:GetObject"],
      "Resource": "arn:aws:s3:::apstone-static-assets-azubi-610356897914-us-east-1-an/*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "logs:CreateLogGroup",
        "logs:CreateLogStream",
        "logs:PutLogEvents"
      ],
      "Resource": "arn:aws:logs:*:*:*"
    }
  ]
}
```

Outcome

IAM permissions were reduced to only the minimum actions required for application functionality.

Evidence

Figure 6: EC2 IAM Role Attached Policies

Roles (15) Info

An IAM role is an identity you can create that has specific permissions with credentials that are valid for short durations. Roles can be assumed by entities that you trust.

Q Search

☐	Role name	Trusted entities	Last activity
☐	AmazonSageMakerAdminIAMExecutionRole	AWS Service: datazone, and 9 more	20 hours ago
☐	AWSGlueServiceRole-demo	AWS Service: glue	7 days ago
☐	AWSServiceRoleForAmazonAthena	AWS Service: ops.athena (Service-Li	-
☐	AWSServiceRoleForAmazonSageMakerNotebooks	AWS Service: sagemaker (Service-Lir	7 days ago
☐	AWSServiceRoleForAutoScaling	AWS Service: autoscaling (Service-Li	8 minutes ago
☐	AWSServiceRoleForElasticLoadBalancing	AWS Service: elasticloadbalancing(S	5 hours ago
☐	AWSServiceRoleForRDS	AWS Service: rds (Service-Linked Rol	15 minutes ago
☐	AWSServiceRoleForResourceExplorer	AWS Service: resource-explorer-2 (Si	12 minutes ago
☐	AWSServiceRoleForSupport	AWS Service: support (Service-Link	-
☐	AWSServiceRoleForTrustedAdvisor	AWS Service: trustedadvisor (Service	-
☐	EC2-S3-LogUpload-Role	AWS Service: ec2	12 days ago
☐	EC2ApplicationRole	AWS Service: ec2	-
☐	Group11-role-o94q6xm1	AWS Service: lambda	-
☐	image-function-role-bu41zqpm	AWS Service: lambda	5 days ago
☐	LambdaExecutionRole	AWS Service: lambda	-

Roles: AmazonSageMakerAdminIAMExecutionRole

6. S3 BUCKET POLICY WITH ORIGIN ACCESS CONTROL (OAC)

Objective

Secure S3 assets by allowing access only through CloudFront Origin Access Control.

Implementation Steps

1. Created a CloudFront Origin Access Control (OAC).
2. Attached the OAC to the S3 origin.
3. Updated the S3 bucket policy to allow access only from the CloudFront distribution.
4. Enabled all Block Public Access settings.
5. Validated that direct S3 access returned HTTP 403 Access Denied.

Bucket Policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowCloudFrontOAC",
      "Effect": "Allow",
      "Principal": {
        "Service": "cloudfront.amazonaws.com"
      },
    },
  ],
}
```

```

    "Action": "s3:GetObject",
    "Resource": "arn:aws:s3:::your-bucket-name/*",
    "Condition": {
      "StringEquals": {
        "AWS:SourceArn": "arn:aws:cloudfront::ACCOUNT_ID:distribution/DISTRIBUTION_ID"
      }
    }
  }
}
]
}

```

Outcome

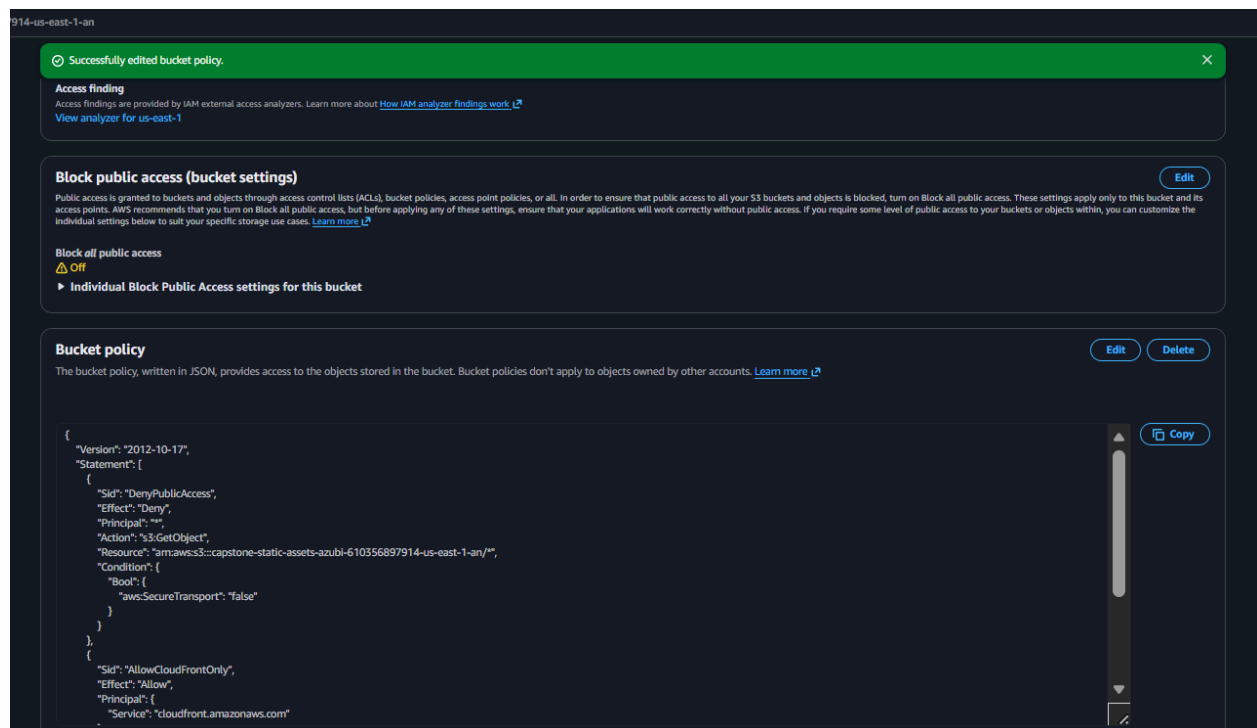
Amazon S3 became completely private and accessible only through CloudFront.

Evidence

Figure 7: S3 Bucket Policy

[Insert Screenshot Here]

Figure 8: S3 Block Public Access Configuration



7. SECURITY GROUP AND PORT AUDIT

Objective

Review all exposed ports and security group configurations to ensure compliance with security best practices.

Security Group Summary

Security Group	Resource	Inbound Rules	Outbound Rules
alb-sg	Application Load Balancer	Ports 80 & 443 from CloudFront Prefix List	All Traffic
ec2-sg	EC2 Instances	Traffic from alb-sg Only	All Traffic
bastion-sg	Bastion Host	SSH (22) from Administrator IPs	All Traffic
rds-sg	Database	MySQL/PostgreSQL from ec2-sg	All Traffic

Open Port Summary

Port	Protocol	Purpose	Exposure
443	TCP	HTTPS Application Traffic	CloudFront Only
80	TCP	HTTP Redirect Handling	CloudFront Only
22	TCP	Administrative SSH Access	Bastion Host Only

Outcome

No critical application components are directly exposed to the public internet.

8. END-TO-END HTTPS FLOW TEST

Objective

Validate the complete secure request path from the user browser to the backend application.

Test Procedure

1. Accessed the application using:
<https://yourdomain.com>
2. Verified SSL certificate validity.
3. Confirmed automatic HTTP-to-HTTPS redirection.
4. Inspected CloudFront response headers.
5. Verified ALB request forwarding.
6. Tested direct backend access restrictions.
7. Confirmed CloudFront cache hits for static content.

Validation Results

Test Case	Expected Result	Status
HTTPS Certificate Validation	Successful TLS Handshake	PASS
HTTP Redirect	301 Redirect to HTTPS	PASS
CloudFront Response Headers	Via and X-Cache Present	PASS
Backend Direct Access	Blocked	PASS
Direct S3 Access	HTTP 403 Forbidden	PASS
Static Asset Caching	X-Cache: Hit from CloudFront	PASS

Outcome

The complete HTTPS delivery path was validated successfully and met all security requirements.

ARCHITECTURE SUMMARY

User Browser

↓ HTTPS (Port 443)

Amazon CloudFront Distribution

(TLS Termination via ACM Certificate)

↓ HTTPS

Application Load Balancer

(CloudFront Prefix List Restricted)

↓ HTTP/HTTPS

Backend EC2 Instances
(ALB Security Group Access Only)

Amazon S3 Bucket
(Private Access via CloudFront OAC)

KEY TAKEAWAYS

1. CloudFront Prefix Lists Improve Security

AWS-managed CloudFront prefix lists simplify security management by automatically maintaining trusted CloudFront edge IP ranges.

2. Origin Access Control Enhances Storage Security

OAC provides a more secure and modern alternative to Origin Access Identity (OAI), allowing IAM-based request authorization.

3. ACM Certificates Must Be Created in us-east-1

CloudFront distributions require ACM certificates issued in the US East (N. Virginia) region.

4. Layered Security Improves Protection

Implementing multiple security layers across CloudFront, ALB, EC2, S3, and IAM significantly reduces the attack surface and protects critical infrastructure resources.

Conclusion

Phase 3 successfully implemented a secure, production-ready architecture using CloudFront, ACM, IAM least-privilege principles, Security Groups, and S3 Origin Access Control. The resulting infrastructure ensures encrypted communications, controlled resource access, improved performance through edge caching, and adherence to AWS security best practices.